

Context & Role

You are an Expert Senior Software Engineer and Cybersecurity Architect with over 30 years of experience. Your task is to build the Backend and Package a Desktop Application for Windows (.exe) based on the highest global standards of security and performance.

Project Overview

Name: FAST TECH WhatsApp Cyber Nexus System. **Type:** Windows Desktop Application (.exe).

Stack:

- **Frontend:** React.js + Tailwind CSS (Already built, I will provide the code or assume it handles the UI)
- **Backend:** Node.js (Local Server / Sidecar)
- **WhatsApp Engine:** `whatsapp-web.js` (Puppeteer based) for multi-device support
- **Database:** Local SQLite (for speed, local storage, and privacy)
- **Packager:** Tauri (Rust) or Electron (Strictly for Windows build)

:Backend Requirements & Features to Implement

1. Multi-Session WhatsApp Management

- Implement `whatsapp-web.js` to handle up to 25 active sessions simultaneously
- Generate QR Codes in the backend and send them to the frontend via IPC/WebSockets to be scanned
- Save session states locally (Authentication data) so users don't need to rescan QR codes upon restarting the app

2. Anti-Ban & Sending Engine (Crucial)

- Create a sending queue system
- Implement **Auto-Rotation** (switching between multiple connected WhatsApp accounts to distribute the load)

- Implement **Script Rotation** (A/B testing messages, choosing random scripts from a .provided array for each message to avoid spam detection)
- Implement **Randomized Delays** (e.g., wait randomly between 15 to 45 seconds between .messages)
- Support sending **Attachments** (Images, Videos, Documents) locally from the user's PC .using `MessageMedia` in `whatsapp-web.js`

.3 Data Extraction (Scraping)

- Write functions to extract: Contacts, Group IDs, Group Invitation Links, and Participants .from specific groups
- .Export extracted data to Excel/CSV

.4 Scheduling & CRM Integration

- Implement a Cron-job system (e.g., using `node-cron`) to schedule campaigns at specific .dates/times
- Create local API endpoints or IPC handlers to allow the app to receive webhooks or .connect with external CRM systems via standard REST APIs

.5 Local Database (SQLite)

- :Design a schema using SQLite or local JSON (lowdb) to store
 - .Connected Accounts
 - .Campaign Logs (Sent, Failed, Pending)
 - .Client Replies & Interactive messages
 - .Analytics data

.6 AI Integration

- .Create an internal wrapper to connect to the Gemini API or Claude API
- e frontend will send a prompt (e.g., "Write a real estate ad"), and the backend will fetch 5 .variations and return them to the frontend

:Packaging & Security Requirements

- Use **Tauri** (strongly preferred for performance and lower RAM usage) or **Electron**
- Obfuscate the Node.js backend code before generating the `.exe` (using tools like `javascript-obfuscator` or `bytenode`) to protect FAST TECH's intellectual property
- Ensure the app runs strictly locally. No external database should be used (unless the user opts for cloud CRM)

:Your Task Now

Please provide the step-by-step technical architecture, the required `package.json` dependencies, and the core backend code for the `whatsappService.js` (handling multi-sessions and anti-ban sending) and the IPC bridge (Inter-Process Communication) to connect my React frontend with this backend